



Faculty of Mathematics and Computer Science

Tehnici de realizare a sistemelor inteligente (TRSI 2024)

Generating Rail Scenes Images for Semantic Segmentation using Text-to-Image Diffusion Models

Răzvan-Gabriel Petec

*Department of Computer Science, Babeş-Bolyai University
I, M. Kogalniceanu Street, 400084, Cluj-Napoca, Romania
E-mail: razvan.petec@stud.ubbcluj.ro*

Abstract

This paper will present a pipeline for enriching some segmentation dataset, consisting of multiple state-of-the-art generative neural networks, and there will be presented the results of this pipeline applied in the context of rail scenes (useful for autonomous trains / trams driving), where there isn't a lot of annotated data. The main models used in this pipeline are Stable Diffusion [10], which is one of the biggest models in text-to-image generative models at the moment; ControlNet [16], an extension of Stable Diffusion with the addition of conditional control (which, in this case, is the segmentation mask), meant to guide the original network in generating images, within the given constraint; and InstructorNet, a novel architecture which takes the idea from ControlNet of locking the production-ready model (Stable Diffusion) and just training a HyperNetwork to influence the weights of the main model, but there is no additional control, so it will just generate some similar images to the ones being trained on (from a smaller dataset, which in this studied case is RailSem19 [15]). In this pipeline, InstructorNet will be used to generate the segmentation maps from an optional structural prompt (which will guide the network the structure of the segmentation masks), which will be passed as control to ControlNet along with an optional environmental prompt (which will guide the network the environmental features of the realistic final image), so in this manner, we created some new pairs (segmentation mask, image) which can be used along the old dataset entries to improve the performance of segmentation models.

© 2024 .

Keywords:

Text-to-Image Diffusion Models; Semantic Segmentation; Rail Scene Images; Computer Vision; Generative AI

1. Introduction

The integration of artificial intelligence (AI) and computer vision technologies into rail transportation systems would be advantageous for enhancing safety, efficiency, and overall operational performance. In this context, semantic

© 2024 .

segmentation, a fundamental task in computer vision, consisting in classifying each pixel in an image to some class, enables understanding the visual environment.

However, the availability of annotated data for training semantic segmentation machine learning models in the rail domain is limited, decreasing the performance of such algorithms, because images which only capture a small fraction of our visual world will make semantic segmentation models not handle well many of the possible situations [5]. In this context one of the biggest datasets is RailSem19 [15] containing 8500 pairs of image-masks, (with some of the annotations also not being that precise), so the motivation behind this research report would be to generate some data in the given scenario and to try to train a semantic segmentation algorithm to prove that some synthetic data would improve the performance of the current state-of-the-art (SOTA) algorithms.

To address this challenge, this paper introduces a novel approach using text-to-image diffusion models to generate synthetic rail scene images specifically for semantic segmentation tasks (so their segmentation mask will be also generated). This is possible due to the recent advancements in generative AI models, particularly text-to-image techniques, which makes possible enriching small annotated datasets by synthesizing realistic images (similar to the ones being trained on and to the ones big models already were trained on) directly from textual descriptions. This innovative technique enables the flexibility to generate diverse and customizable datasets, containing various scenarios in various environmental conditions encountered by trains / trams.

The proposed problem is a generational one, so we need some models that translate some kinda input into our desired output. Because translating from a plain text to both a realistic image and its segmentation mask would be a challenging task for one model and for the human as well (to encapsulate all the desired data into a given prompt), this paper proposes a 2 step generation algorithm: one that translates structural prompts into segmentation masks (ex: an environment specific to a tramway, 10 people on the cross walk, 3 cars on the road and near another tram circulating in the opposite way), and one that takes as input some environmental prompt and a segmentation mask and generates a realistic image (ex. the tram should be near a mountain and there should be some snow on the rails). Later on, the model could be used without any input (so the output would be kinda random) if only training data is needed to be generated.

2. Related Work

2.1. Generative Adversarial Models

Deep generative models have shown promising results in modeling image distribution, thus enabling synthesis of realistic images. Tritrong et. al [14] used a Generative Adversarial Network (GAN) to generate realistic images of faces and a Few-Shot Segmenter to generate their corresponding masks, which resulted in pairs of (image, segmentation mask) which were used to train an Auto-Shot Segmenter to predict those segmentation masks (Fig. 1), the resulting accuracy of the Auto-Shot Segmenter (smaller and faster model) being close to the one obtained by the Few-Shot Segmenter (bigger and slower model).

Their objective was to take a very small dataset, and to train with as little human effort as possible an Auto-Shot Segmenter that works well in various scenarios. However, this approach has drawbacks, including the considerable human effort required for data annotation and the limited robustness of their GAN model, which fails to understand a wide range of scenarios necessary for training a Rail Scenes Segmenter. This limitation becomes particularly significant in scenarios where the consequences of incorrect predictions are bigger.

2.2. Latent Diffusion Models

The Latent Diffusion Models (LDMs) were firstly introduced by Rombach et al. [10], and they achieve state-of-the-art image generation results by encoding text inputs into latent vectors via pretrained language models like CLIP [9]. During training, they apply Gaussian noise to the image, and the objective of such networks is to denoise the noisy image. The input of such network is the noisy image, the encoded timestamp T (which represents how many times the Gaussian noise was applied to the image) and the encoded text, and the output will be an approximation of the initial image (the direct output of the model is the approximation at step $T - 1$, but this process is applied T times, with the new timestamp as an input, to let the model "know" how far it is from the real output). During inference, a random

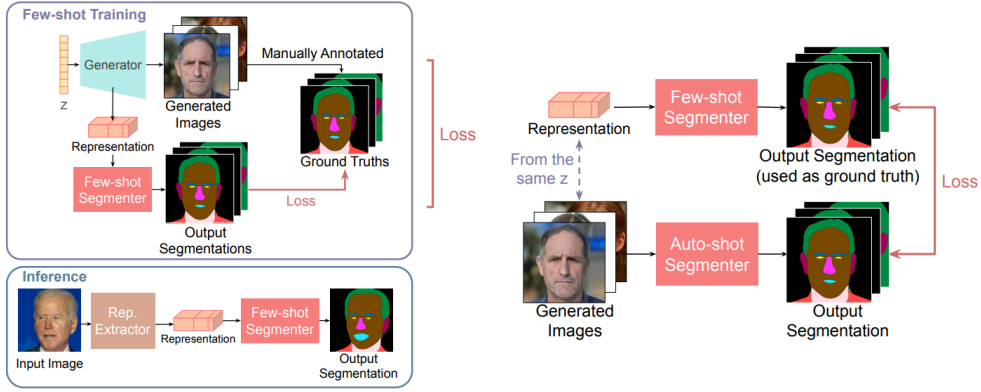


Fig. 1: Few-shot segmentation pipeline and Auto-shot segmentation pipeline

noise from the normal distribution is given to the model, along the text encodings and a given encoded timestamp, and the model tries to predict what the original image was (which would be completely random given the fact that the input was random), then, Gaussian noise is applied to that input less times than the previous timestamp and the new image is given back to the network. This process repeats until the timestamp becomes 0, usually obtaining a good looking image, if the model was trained on a lot of data.

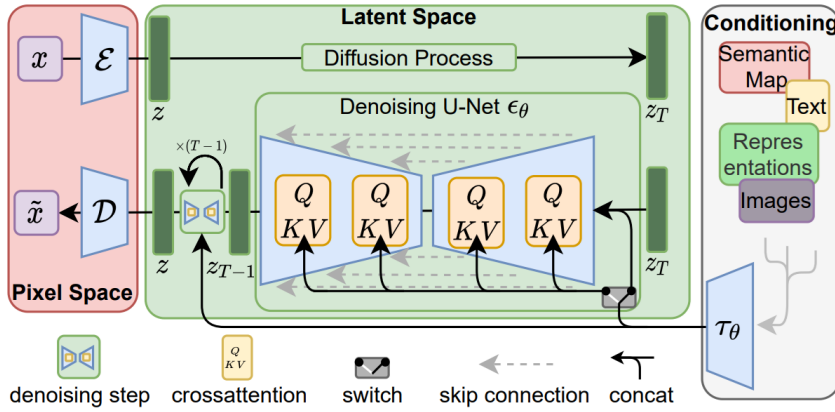


Fig. 2: The architecture of Stable Diffusion

Specifically, Stable Diffusion, the model trained by Romback et. al [10], was trained on the LAION-5B dataset [13], which contains 5.8 billion image-text pairs. A disadvantage of fine-tuning such a big model is that, found out by Perera et al. [8], LDMs tend to generate more replications when they are being trained with a smaller numbers of samples, and given the small size of the datasets in the proposed context, this could not be a reliable solution.

2.3. ControlNet

Proposed by Zhang et al. [16], ControlNet is a HyperNetwork [2] (a smaller network trained to influence the weights of a larger network) applied on some LDM (Stable Diffusion in this case). The purpose of this network is to add some kind of conditional control (e.g. segmentation masks, canny edges, human poses) to the locked production-ready model.

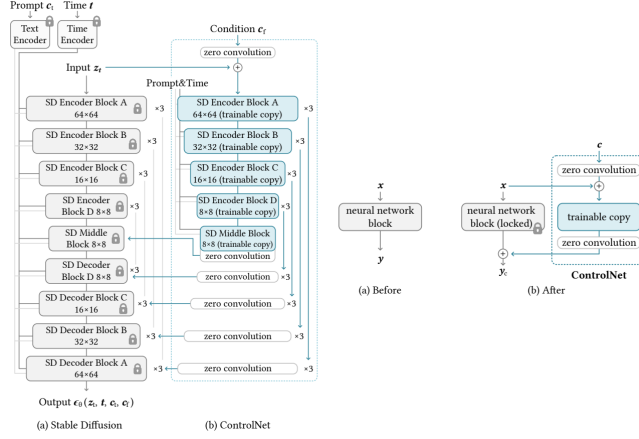


Fig. 3: The architecture of ControlNet (left) and a Neural Block (right)

The main challenge that they are trying to solve is training a model on a way smaller dataset with the help of the big model and to avoid overfitting and catastrophic forgetting (which they state in the paper that is a problem which happens when the big model is just fine-tuned).

The proposed model locks the production-ready large diffusion model and reuses their deep and robust encoding layers pretrained with billions of images as a strong backbone to learn a diverse set of conditional controls. The neural architecture is connected with "zero convolutions" (zero-initialized 1x1 convolution layers) that progressively grow the parameters from zero and ensure that no harmful noise could affect the finetuning.

Having a pretrained neural block (Fig. 3.2a), a ControlNet block could be added by locking (freezing) the parameters of the original block and simultaneously cloning the block to a *trainable copy* (Fig. 3.2b). The trainable copy will receive an external conditioning vector c as input. When such structure is applied to large models like Stable Diffusion, the locked parameters preserve the original model trained on billions of images, while the trainable copy will reuse the original model to establish a deep, robust, and strong backbone for handling diverse input conditions.

This architecture looks more promising than the other ones, enabling us to "fine-tune" a big model in such a way that it will have the robustness of the production-ready model and also do our specific task (under mentioned constraints). The problem with this approach in the proposed problem is that we don't have yet any constraint under which the model will generate the output. We can use the already existing masks from our small dataset, but, in that way, we could not control the generated structure of the image in any way, which could lead our semantic segmentation model to handle easy situations well, but struggle with more unusual ones.

3. Approach

3.1. Dataset

Proposed by Zendel et al., RailSem19 [15] is the one of the biggest datasets in the proposed context. It is a multi-purposed dataset, containing multiple trains / trams images in various scenarios. For segmentation, it contains 8500 (image, mask) pairs. The mask contains gray pixels ranging from 0 to 18, and the 255 value, the significations of each pixels is the following:

- the first 19 pixels represents the following classes: *road, sidewalk, construction, tram-track, fence, pole, traffic-light, traffic-sign, vegetation, terrain, sky, human, rail-track, car, truck, trackbed, on-rails, rail-raised, rail-embedded*
- the pixels with the value 255 represents the class *void*



Fig. 4: Examples of entries in the RailSem19 dataset

3.2. Preprocessing

3.2.1. Basic transformations

The dataset contains image-mask pairs with the resolution $1920 \times 1080 \times 3$, and the models usually takes as input and gives as output some $512 \times 512 \times 3$ image, so the central crop was taken of both the mask and the image (Fig. ??), and the image was resized to the resolution 512×512 using Area-based Resampling, and the mask was resized to the same resolution using Nearest Neighbours Method (to be sure the pixels weren't mixed up).

3.2.2. Prompts

Because the dataset contains only pairs of (image, segmentation mask) and the proposed models can receive as input some text (as proposed, we need a structural prompt, used for generating the semantic mask, and an environmental prompt, used for generating the realistic image, along the semantic mask).

The pre-trained model BLIP-2 [4] was used to extract textual descriptions of the images in dataset. This model receives as input an image and some optional prompt, then it would try to continue the prompt based on the given image. In this way, this model can be used to extract some information about the given images.

In the experiments, multiple models and multiple prompts were tested, the model that worked the best was *blip2-opt-2.7b-coco*, which was trained on the images from the COCO dataset [5], which also contains some Rail scenes that could've been beneficial. The best textual descriptions of the images were obtained by giving no prompt to the dataset, and those descriptions were used for the structural prompt and for the environmental prompts. To each of the prompts obtained by BLIP2, the text ", high quality, extremely detailed, 4K, HQ" was appended (similar to how they did in the ControlNet paper).

Other kinds of prompting used in the experiments was with no prompt at all (because it's optional, it will learn only from the images), and with a fixed prompt for all the entries (e.g. "train tracks high-quality image").

Also, a fixed negative prompt was used for some of the experiments. This only affected the inference step. The negative prompt can be saw in Fig. 5.

```
negative_prompt = (
    "\"accidents, derailments, overgrown tracks, overgrown stations, impossible curves, missing switches, \"
    \"double rail blade, cloudy weather, \"
    \"cartoon, cgi, render, 3d, artwork, illustration, 3d render, cinema 4d, artstation, octane render, painting, \"
    \"oil painting, anime, 2d, sketch, drawing, bad photography, bad photo, deviant art, overexposed, simple background, \"
    \"plain background, grainy, portrait, grayscale, monochrome, underexposed, low contrast, low quality, dark, distorted, \"
    \"white spots, deformed structures, macro, multiple angles, nsfw, uncensored, nude, ugly, bad anatomy, bad proportions, \"
    \"blurry, cropped, deformed, disconnected limbs, out of frame, out of focus, dehydrated, error, disfigured, \"
    \"disgusting, extra arms, extra limbs, extra hands, fused fingers, gross proportions, long neck, low res. jpeg, \"
    \"jpeg artifacts, malformed limbs, mutated, mutated hands, mutated limbs, missing arms, missing fingers, picture frame, \"
    \"poorly drawn hands, poorly drawn face, text, signature, username, watermark, worst quality, collage, pixel, pixelated\""
)
```

Fig. 5: The negative prompt used for inferences

3.2.3. Injecting labels significations into the masks

In the original dataset, as described in Section 3.1, the labels were numbers with a predefined signification, but this isn't beneficial when generating the segmentation masks, because the model should also learn the signification of the labels, without any hint. Also, if the model outputs a value between (k, k, k) and $(k + 1, k + 1, k + 1)$, it would be hard to differentiate if it belongs to class k or to class $k + 1$, or even harder if it outputs a value in the middle of the interval $[(18, 18, 18) \dots (255, 255, 255)]$.

Table 1: Color Mappings from word2vec vectors with the dimensionality reduced with PCA

Label	RGB	Hex
road	(109, 99, 95)	#6D635F
sidewalk	(102, 126, 185)	#667EB9
construction	(139, 118, 91)	#8B765B
tram-track	(255, 3, 92)	#FF035C
fence	(89, 90, 240)	#595AF0
pole	(62, 155, 255)	#3E9BFF
traffic-light	(113, 112, 90)	#71705A
traffic-sign	(105, 100, 125)	#69647D
vegetation	(22, 45, 56)	#162D38
terrain	(55, 84, 0)	#375400
sky	(0, 126, 87)	#007E57
human	(66, 139, 59)	#428B3B
rail-track	(124, 109, 95)	#7C6D5F
car	(122, 214, 131)	#7AD683
truck	(141, 243, 123)	#8DF37B
trackbed	(131, 95, 103)	#835F67
on-rails	(252, 255, 40)	#FCFF28
rail-raised	(90, 136, 98)	#5A8862
rail-embedded	(187, 0, 140)	#BB008C
void	(255, 255, 255)	#FFFFFF

That's why the model word2vec [7] was used to generate vector values for each label. Because each vector were from R^{300} (so vectors with 300 real values), Principal Component Analysis (PCA) [6] was used to reduce the dimensionality to 3, so the new vectors would belong to R^3 , then, min-max scaling was used to map the values in the interval $[0, 255]^3$, then each value was rounded to the closest value and that's how the color mapping from Table 1 was obtained.

3.3. InstructorNet

The architecture of this neural network takes the idea of a HyperNetwork [2], so instead of fine-tuning a big model, just create another network to influence its weights such that it will solve the given problem. The proposed approach is the same as ControlNet [16], but with the removal of the condition c as an input, so the trainable copy of the network will receive as input directly the input z from the latent space, and will guide the network to output some image similar with the ones being trained on, but keeping the production-ready model locked to be able to generate various scenarios which the original model experienced in the original dataset.

3.4. Assembling the pipeline

So, the proposed pipeline for the task of enriching semantic segmentation datasets (consisting in two sets: S - the set of semantic segmentation masks, and R - the set of realistic images) is the following: taking an optional structural prompt, it will be passed through InstructorNet and a new semantic segmentation mask s will be generated, s will be

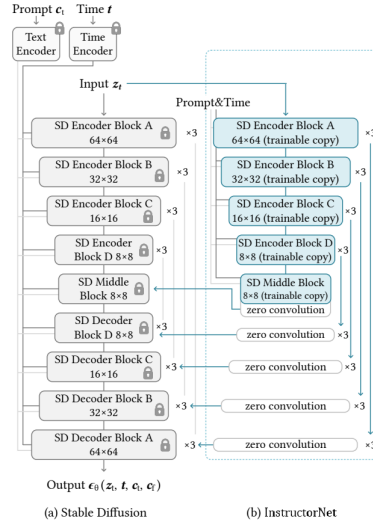


Fig. 6: The architecture of InstructorNet, as you can see, is heavily inspired by the one from ControlNet (Figure 3.1)

passed through ControlNet along with an optional environmental prompt, which will result in a realistic image r under the constraints of s . These images will enrich our semantic segmentation dataset (so $S \leftarrow S \cup \{s\}$ and $R \leftarrow R \cup \{r\}$). Then, to train a semantic segmenter (Φ), we would take two matching samples $a \in S$ and $b \in R$, then would compute the predicted image $\Phi(b) = a'$, then compute the loss as the mean square error between each pixel from a and a' , which will be backpropagated through Φ in the process of learning. The whole pipeline is also illustrated in Fig. 7.

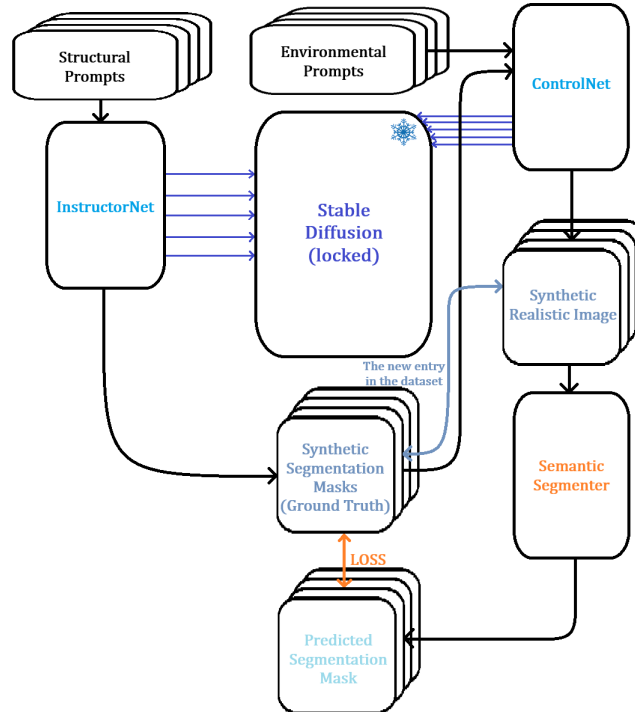


Fig. 7: An illustration of the pipeline

3.5. Metrics

For measuring the quality of the generated images, the metric used was Fréchet inception distance (FID), introduced by Heusel et al. [3]. Unlike inception score (IS), introduced by Salimans et al. [12], which is widely used in the context of generating images and measures mainly the distribution of the generated images, FID compares the distribution of the generated images with the ones from the real images ("ground truth").

For two multidimensional Gaussian distributions $N(\mu, \Sigma)$ and $N(\mu', \Sigma')$, their FID is explicitly solvable as:

$$d_F(N(\mu, \Sigma), N(\mu', \Sigma'))^2 = \|\mu - \mu'\|^2 + \text{tr}(\Sigma + \Sigma' - 2(\Sigma\Sigma')^{\frac{1}{2}}),$$

where $N(\mu, \Sigma)$ is the Gaussian distribution for $f(S)$ and $N(\mu', \Sigma')$ is the Gaussian distribution for $f(S')$, S and S' being the real dataset, respectively the synthetic dataset, and f being usually chosen as an Inception v3 model trained on ImageNet, without its final classification layer, which outputs a 2048-dimensional activation vector (Fig. 8).

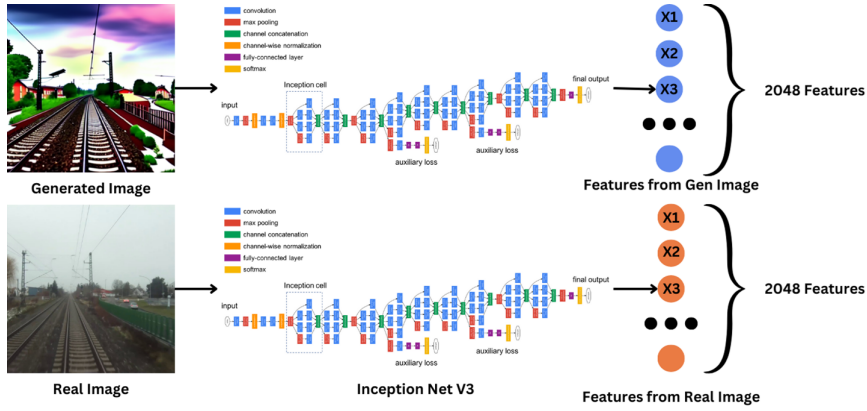


Fig. 8: How features are extracted using FID

4. Results

4.1. Experiments Setup

All these experiments were done on a Nvidia GeForce GTX 1080Ti 12GB GPU. The dataset containing 8500 images was splitted into training data (6800 image-mask pairs) and validation data (1700 image-mask pairs). Each model was then trained for 13 epochs (22100 steps), and the results shown are the ones from the validation dataset. Batch size 4 was used during training as the size of the GPU didn't allow a bigger one.

4.2. ControlNet results

For these experiments, different kind of text prompts were tried to see how it affects the performance. The environment of the experiments is the same as the one described at Section 4.1. The results experimenting with these conditions can be seen in Fig. 9. Also, the logs about these experiments were logged in [this wandb report](#). The learning rate used for these ones was also 10^{-5} . There were some experiments with different learning rates done (10^{-5} , 10^{-4} , $4 \cdot 10^{-5}$, $4 \cdot 10^{-4}$ – which crashed, so a big enough learning rate will crash the experiments) and they were logged in [this other wandb report](#). Each experiment for this model took 20 hours to complete.

FID was measured for each of the condition experiments, and a table with the results can be seen in Table 2.

A comparison between using negative prompts and not can be seen between Fig. 10 (without the negative prompts) and Fig. 11 (with the negative prompts).



Fig. 9: Results of different prompts applied

Table 2: FID Scores for Different Methods

Method	Original masks	Canny
No prompt	76.70	80.07
Fixed prompt	77.77	79.19
BLIP2 prompt	76.72	78.93
Negative prompt	77.12	81.37
Fixed + Negative prompt	77.28	80.95
BLIP2 + Negative prompt	76.57	81.35

4.3. InstructorNet results

This model took under the conditions described at Section 4.1 20 hours for the whole training. Inferences from 1000 to 1000 steps were logged, also, a checkpoint for the model was taken each of the steps an inference was done. In Table 3 we can see some results from the last epoch (the 22100th one). The learning rate used for this experiment was 10^{-5} . The results can be also seen in [this wandb report](#). The experiment for this model took 20 hours to complete.

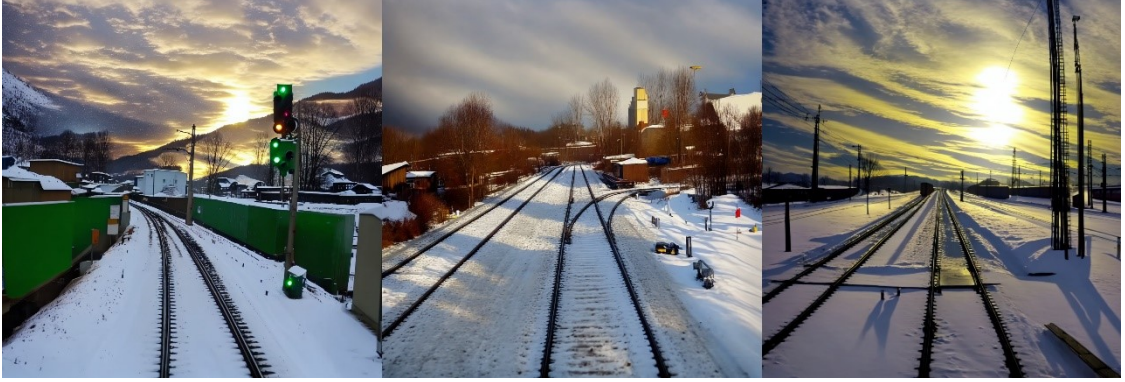


Fig. 10: Results obtained using original masks with BLIP2 prompts. Images from left to right: 8469, 8482, 8499.



Fig. 11: Results obtained using original masks with BLIP2 prompts and negative prompts. Images from left to right: 8469, 8482, 8499.

4.4. Semantic Segmentation Results

The trained segmentation model was the U-Net [11] neural network architecture. This architecture is suitable for high-resolution semantic segmentation tasks, as it has skip-connections between the encoder and decoder blocks to ensure both information and location flow.

The choice of the network was inspired by the competitive results obtained in [1]. The model was trained for 40 epochs at each step using the Adam optimizer with a learning rate of $1e-4$ and batch size 4. The images are resized to 512×512 . Only the best validation performance achieved at each step was reported.

Multiple setups were considered, named A, B, C, D and E. In setup A, the training set consists of 3000 real images and 500 test images. The ids for these images are: 0-2999 for train, and 3000-3499 for test. In setup B, 3000 synthetic images (ids 0-2999) were considered and the same 500 test images (ids 3000-3499). In setup C, the real and synthetic images were combined, obtaining 6000 train images (3000 real, 3000 synthetic), maintaining the same 500 images for testing. Setup D is similar to setup C, with the mention that only 300 images are selected in total for training, 1500 real and 1500 synthetic. Setup E is similar to setup A, but instead of 3000 training images there are 6000. All synthetic images were obtained using the following training setup: 6800:1700 train:validation split, grayscale images, BLIP2 prompts, batch size 16, 40 epochs, $1e-4$ learning rate, image size 512×512 .

Intersection over Union (IoU) metric was used to assess the performance of the trained models. Since the work was done in a binary context where the rails (positive class) are segmented, the focus will be on the mean IoU score for both classes (positive and negative).

Each experiment was performed three times. The mean and standard deviation obtained for the mIoU score were reported in Table 4.

Table 3: Some of the results of InstructorNet

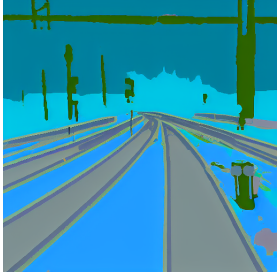
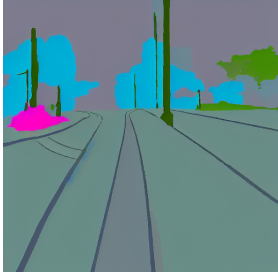
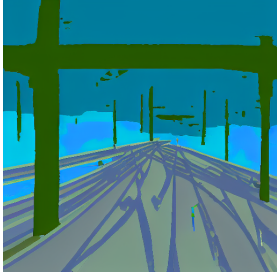
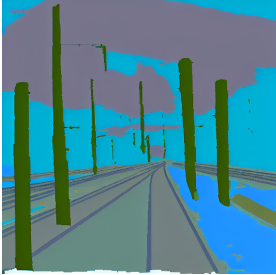
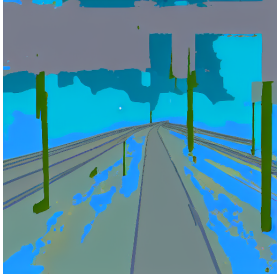
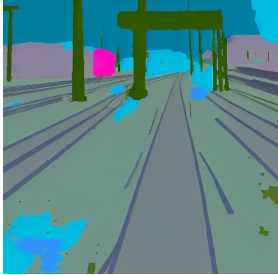
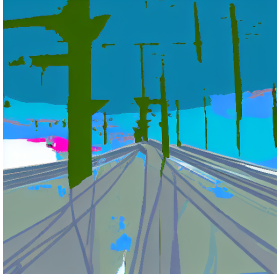
Good Results		Average Results	Bad Results
<i>"a view of a train track with a few trees in the background"</i>	<i>"a train track with snow on it"</i>	<i>"a train track with a crossroads on it and a building in the background"</i>	<i>"a train track with snow on it and wires hanging from the sky"</i>
			
			

Table 4: Semantic segmentation of rails results comparison with the three different setups considered.

Setup	mIoU	Std. Deviation
A	78.493	0.293
B	78.295	0.546
C	80.208	0.542
D	78.369	0.253
E	80.268	0.686

5. Discussion

5.1. Results interpretation

We can see promising results on the results for both the models. From the experiments done on the learning rate at Section 4.2, we can see that the best looking results were obtained for learning rate 10^{-5} , that's why it was used on the other experiments too.

The highest FID score is obtained by combining BLIP2 prompts with Negative prompts using the original grayscale masks. Visually, the images do indeed look the most realistic out of all other configurations. Some examples are displayed in Fig. 11.

For semantic segmentation (Section 4.4), setups A and E imply training only on real images. By training on 3000 images (setup A), the mIoU score reaches 78.493. Training on 6000 images (setup E), the mIoU becomes 80.268. If out of the 6000 images (from setup E), half of the images were replaced with their synthetic alternatives (setup C), the performance stays the same, with a small decrease in mIoU and also in standard deviation. If out of the 3000 images from setup A half of the images were also replaced with synthetic images (setup D), the performance again does not modify significantly.

In other words, adding synthetic images instead of real ones does not increase the performance (see setups A vs D, and C vs E). However, enriching the training set with synthetic images on top of the real ones does increase the performance. For proof of this, consider setup C. This supports the idea that synthetic images are beneficial for training.

5.2. Applications of the pipeline

Some applications of the proposed pipeline would be:

- **Autonomous Trains Development:** Those synthetic pairs of image-mask generated from text descriptions can be used to train and test semantic segmentation models for autonomous vehicles. This enables the development and validation of perception systems in a controlled environment before real-world deployment, enhancing safety and reliability.
- **Railway Maintenance and Inspection:** Semantic segmentation models trained on synthetic images can assist in the automatic detection and classification of defects or anomalies in railway infrastructure, such as cracks in tracks, damaged signals, or vegetation encroachment. This can streamline maintenance operations and improve overall safety.
- **Training Simulators for Railway Operators:** Synthetic rail scene images can be utilized to develop training simulators for railway operators and personnel. By providing realistic scenarios, operators can practice various tasks such as signal recognition, obstacle detection, and emergency response in a safe and controlled virtual environment.

5.3. SWOT Analysis

Strengths:

- **High-Quality Image Generation:** Text-to-image diffusion models are known for their ability to create realistic and diverse images. This can be beneficial for generating a large and varied dataset of rail scene images for semantic segmentation tasks.
- **Open-Ended Scene Description:** Text prompts allow for detailed descriptions of the scene, including specific objects, weather conditions, and lighting. This flexibility can lead to a more comprehensive dataset compared to traditional methods.
- **Potential for Unseen Categories:** The model might be able to handle object categories not present in the training data, due to the reliance on textual descriptions.

Weaknesses:

- **Accuracy and Control:** Diffusion models might struggle with precise segmentation, especially for small objects or complex scenes. Text prompts may not fully capture the intricacies needed for accurate segmentation.
- **Limited Training Data Availability:** Training text-to-image diffusion models requires a vast amount of data. Specialized datasets for rail scenes with corresponding text descriptions may be scarce.
- **Computational Cost:** Training and running diffusion models can be computationally expensive, requiring powerful GPUs.

Opportunities:

- **Focus on Safety-Critical Objects:** Semantic segmentation of rail scenes can be crucial for applications like automated train inspection, focusing on accurate segmentation of tracks, signals, and other safety-critical objects.
- **Domain-Specific Training:** Fine-tuning the diffusion model on existing, labeled rail scene datasets can improve its performance in this specific domain.
- **Exploration of Alternative Techniques:** Techniques like combining diffusion models with traditional segmentation methods might lead to better segmentation accuracy.

Threats:

- **Evolving Regulations:** Regulations around data privacy and security in the rail industry might limit data collection for training the model.
- **Alternative Approaches:** Other techniques for semantic segmentation, such as encoder-decoder architectures, might prove more efficient or accurate for this specific task.

6. Conclusions and future work

In conclusion, this pipeline is extremely promising and deserve further attention which would best be focused on experimenting with multiple hyperparameters, especially with the InstructorNet architecture. Also, the capabilities of the used metric (FID) could be revisited and addressed. It is not clear that InceptionNetV3 is capable to encode rail-specific knowledge thus, a more specialized backbone could be used to compute a more domain accurate Fréchet Distance.

Acknowledgments

During the preparation of this work the author(s) used Gemini (from Google) in order to write Sections 5.2 and 5.3. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

References

- [1] Alexandrescu, A.R., Manole, A., 2022. A dynamic approach for railway semantic segmentation. *Studia Universitatis Babeş-Bolyai Informatica* 67, 61–76. URL: <https://www.cs.ubbcluj.ro/~studia-i/journal/article/view/79>, doi:10.24193/subbi.2022.1.05.
- [2] Ha, D., Dai, A., Le, Q.V., 2016. Hypernetworks. [arXiv:1609.09106](https://arxiv.org/abs/1609.09106).
- [3] Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S., 2018. Gans trained by a two time-scale update rule converge to a local nash equilibrium. [arXiv:1706.08500](https://arxiv.org/abs/1706.08500).
- [4] Li, J., Li, D., Savarese, S., Hoi, S., 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. [arXiv:2301.12597](https://arxiv.org/abs/2301.12597).
- [5] Lin, T.Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P., 2015. Microsoft coco: Common objects in context. [arXiv:1405.0312](https://arxiv.org/abs/1405.0312).
- [6] Maćkiewicz, A., Ratajczak, W., 1993. Principal components analysis (pca). *Computers Geosciences* 19, 303–342. URL: <https://www.sciencedirect.com/science/article/pii/009830049390090R>, doi:[https://doi.org/10.1016/0098-3004\(93\)90090-R](https://doi.org/10.1016/0098-3004(93)90090-R).
- [7] Mikolov, T., Chen, K., Corrado, G., Dean, J., 2013. Efficient estimation of word representations in vector space. [arXiv:1301.3781](https://arxiv.org/abs/1301.3781).
- [8] Perera, M.V., Patel, V.M., 2023. Analyzing bias in diffusion-based face generation models. 2023 IEEE International Joint Conference on Biometrics (IJCB), 1–10 URL: <https://api.semanticscholar.org/CorpusID:258615767>.
- [9] Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I., 2021. Learning transferable visual models from natural language supervision. [arXiv:2103.00020](https://arxiv.org/abs/2103.00020).
- [10] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B., 2022. High-resolution image synthesis with latent diffusion models. [arXiv:2112.10752](https://arxiv.org/abs/2112.10752).
- [11] Ronneberger, O., Fischer, P., Brox, T., 2015. U-net: Convolutional networks for biomedical image segmentation. [arXiv:1505.04597](https://arxiv.org/abs/1505.04597).
- [12] Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., Chen, X., 2016. Improved techniques for training gans. [arXiv:1606.03498](https://arxiv.org/abs/1606.03498).
- [13] Schuhmann, C., Beaumont, R., Vencu, R., Gordon, C., Wightman, R., Cherti, M., Coombes, T., Katta, A., Mullis, C., Wortsman, M., Schramowski, P., Kundurthy, S., Crowson, K., Schmidt, L., Kaczmarczyk, R., Jitsev, J., 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. [arXiv:2210.08402](https://arxiv.org/abs/2210.08402).
- [14] Tritrong, N., Rewatbowornwong, P., Suwajanakorn, S., 2021. Repurposing gans for one-shot semantic part segmentation. [arXiv:2103.04379](https://arxiv.org/abs/2103.04379).
- [15] Zendel, O., Murschitz, M., Zeilinger, M., Steininger, D., Abbasi, S., Beleznaï, C., 2019. Railsem19: A dataset for semantic rail scene understanding, pp. 1221–1229. doi:10.1109/CVPRW.2019.00161.
- [16] Zhang, L., Rao, A., Agrawala, M., 2023. Adding conditional control to text-to-image diffusion models. [arXiv:2302.05543](https://arxiv.org/abs/2302.05543).